

Krzywa pogoni

Modelowanie matematyczne

Skowroński Piotr Czuba Krzysztof

Informatyka, studia II stopnia (sem. II)

11 stycznia 2026

Plan prezentacji

- 1 Wprowadzenie
- 2 Strategie pościgu
- 3 Rozszerzenia geometryczne
- 4 Implementacja
- 5 Wnioski

Czym jest problem pościgu?

Problem: Określenie trajektorii ścigającego, który dąży do przechwycenia poruszającego się celu.

Zastosowania:

- Naprowadzanie rakiet i pocisków
- Roboty autonomiczne i drony
- Nawigacja morska i lotnicza
- AI w grach komputerowych
- Optymalizacja trajektorii

Podejście: Rozwiązywanie równań różniczkowych opisujących ruch ścigającego i celu.

Pościg bezpośredni (Direct Pursuit)

Ścigający w każdej chwili celuje **bezpośrednio** w aktualną pozycję celu.

Równanie ruchu:

$$\frac{d\vec{p}}{dt} = v_p \cdot \frac{\vec{t} - \vec{p}}{\|\vec{t} - \vec{p}\|}$$

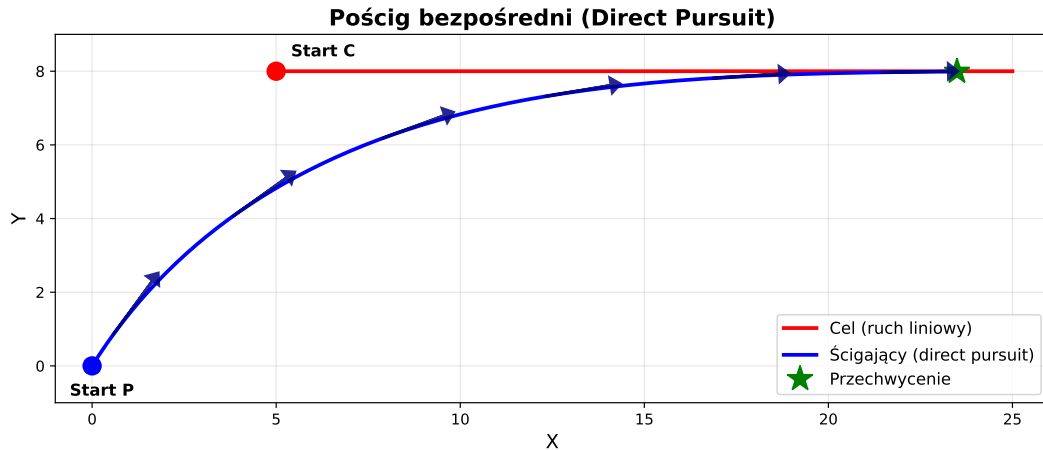
gdzie:

- $\vec{p}$ – pozycja ścigającego
- $\vec{t}$ – pozycja celu
- v_p – prędkość ścigającego

Warunek przechwycenia: $v_p > v_t$ (ścigający szybszy od celu)

Pościg bezpośredni – wizualizacja

Parametry: $v_p = 1.3 \text{ m/s}$, $\vec{v}_t = (1.0, 0) \text{ m/s}$



Stały kąt namiarowania (Constant Bearing)

Ścigający porusza się pod **stałym kątem** β względem kierunku do celu.

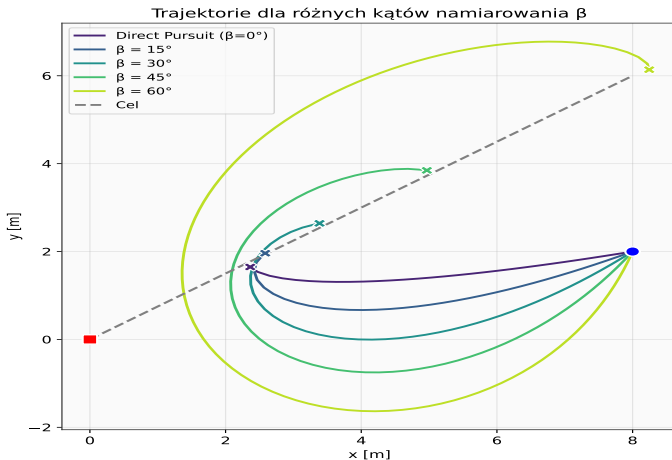
Równania:

$$\begin{aligned}\theta_{\text{target}} &= \arctan 2(t_y - p_y, t_x - p_x) \\ \theta_{\text{pursuer}} &= \theta_{\text{target}} + \beta\end{aligned}$$

Trajektoria tworzy **spirale logarytmiczną**.

Stały kąt namiarowania – wizualizacja

Parametry: $v_p = 2.0$ m/s, $\vec{v}_t = (0.8, 0.6)$ m/s



Nawigacja proporcjonalna (Proportional Navigation)

Strategia szeroko stosowana w **naprowadzaniu pocisków**.

Kąt LOS (Line of Sight):

$$\lambda = \arctan 2(t_y - p_y, t_x - p_x)$$

Zmiana kąta LOS:

$$\Delta\lambda = \lambda(t) - \lambda(t - \Delta t)$$

Aktualizacja kierunku ścigającego:

$$\theta_{\text{new}} = \theta_{\text{current}} + N \cdot \Delta\lambda$$

gdzie N – stała nawigacji (typowo 3–5). **Optymalna wartość:** $N \in [3, 4]$.

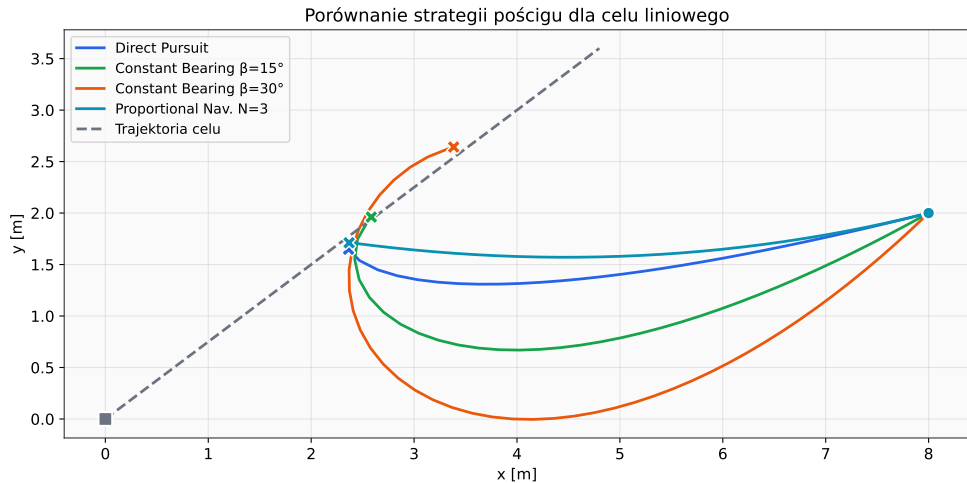
Porównanie strategii – wyniki liczbowe

Strategia	Parametr	Wydłużenie trajektorii	Stabilność
Direct Pursuit	–	1.0 (baseline)	Stabilna
	$\beta = 15$	1.098 ± 0.05	Stabilna
Constant Bearing	$\beta = 30$	1.438 ± 0.05	Stabilna
	$\beta = 45$	2.185 ± 0.05	Średnia
	$N = 2$	0.992 ± 0.03	Wysoka
Proportional Nav.	$N = 3$	0.991 ± 0.03	Wysoka
	$N = 5$	0.990 ± 0.03	Średnia

Wniosek: Nawigacja proporcjonalna daje najkrótsze trajektorie, ale Direct Pursuit jest prostszy w implementacji.

Porównanie strategii – cel liniowy

Parametry: $v_p = 2.0$ m/s, $\vec{v}_t = (0.8, 0.6)$ m/s



Analiza stosunku prędkości $\rho = v_p/v_t$ - cel na okręgu (Direct Pursuit)

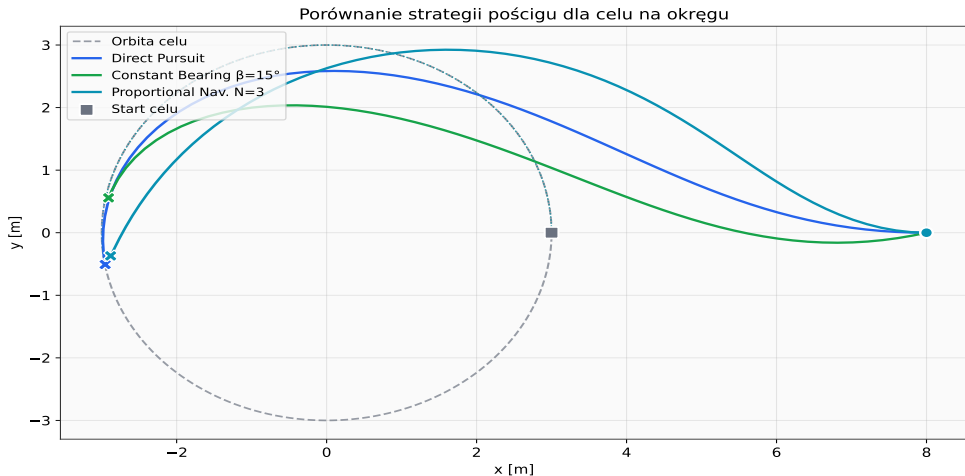
Stosunek $\rho = v_p/v_t$	Czas zbieżności	Status
$\rho < 1.0$	∞	Brak przechwycenia
$\rho = 1.1$	$0.81 \cdot T_{\text{orbit}}$	Powolna zbieżność
$\rho = 1.5$	$0.35 \cdot T_{\text{orbit}}$	Efektywne przechwycenie
$\rho = 2.0$	$0.19 \cdot T_{\text{orbit}}$	Szybkie przechwycenie
$\rho \gg 1$	$\approx d_0/v_p$	Zbieżność do ruchu prostoliniowego

gdzie $T_{\text{orbit}} = 2\pi r/v_t$ – okres orbitowania celu, d_0 – początkowa odległość ścigający-cel.

Krytyczna prędkość: Dla celu na okręgu o promieniu r z ω : $v_p > r \cdot \omega = v_t$

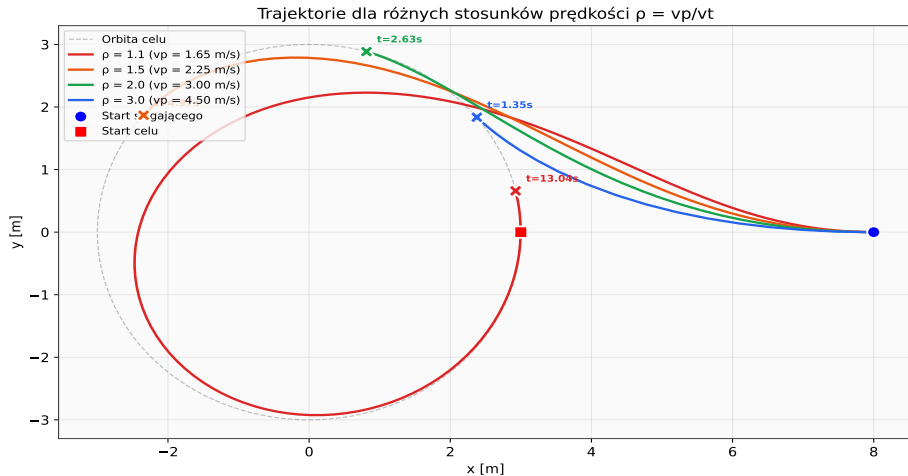
Porównanie strategii – cel na okręgu

Parametry: $r = 3$ m, $\omega = 0.5$ rad/s, $v_p = 2.0$ m/s

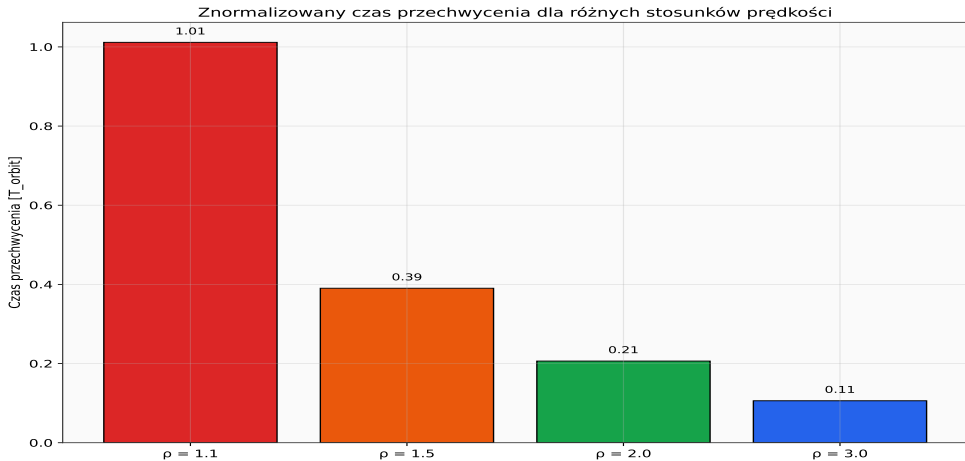


Stosunek prędkości – trajektorie (Direct Pursuit)

Parametry: $r = 3$ m, $\omega = 0.5$ rad/s, $v_t = 1.5$ m/s



Stosunek prędkości – czasy przechwycenia



Pościg cykliczny (Cyclic Pursuit)

n agentów na okręgu o promieniu R , każdy ściga kolejnego: $\vec{p}_{n+1} = \vec{p}_1$

Równanie ruchu agenta i :

$$\frac{d\vec{p}_i}{dt} = v \cdot \frac{\vec{p}_{i+1} - \vec{p}_i}{\|\vec{p}_{i+1} - \vec{p}_i\|}$$

Wyprowadzenie czasu zbieżności:

- ❶ Kąt środkowy między sąsiednimi agentami: $\alpha = \frac{2\pi}{n}$
- ❷ Kierunek ruchu agenta i tworzy z promieniem kąt: $\theta = \frac{\pi}{2} - \frac{\alpha}{2} = \frac{\pi}{2} - \frac{\pi}{n}$
- ❸ Składowa prędkości ku centrum: $v_r = v \cdot \cos \theta = v \cdot \sin\left(\frac{\pi}{n}\right)$
- ❹ Czas zbieżności: $T = \frac{R}{v_r} = \frac{R}{v \cdot \sin(\pi/n)}$

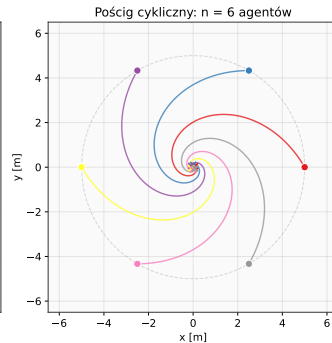
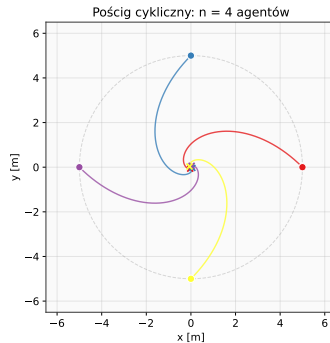
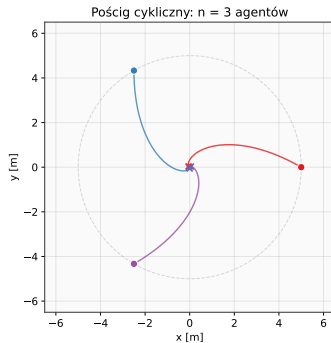
Liczba agentów n	T_{theory} [R/v]	$T_{\text{numerical}}$ [R/v]	Błąd względny [%]
3	1.155	1.153 ± 0.01	0.12
4	1.414	1.412 ± 0.01	0.14
6	2.000	1.996 ± 0.01	0.20
12	3.864	3.849 ± 0.01	0.39
18	5.759	5.726 ± 0.01	0.58

Uwaga: Błąd wynika z warunku stopu symulacji (odległość < 0.01 m).

$$\text{Asymptotycznie: } \lim_{n \rightarrow \infty} T_{\text{conv}}(n) = R/v$$

Pościg cykliczny – wizualizacja

Parametry: $R = 5$ m, $v = 1.0$ m/s



Pościg w 3D – trajektoria helisoidalna (Direct Pursuit)

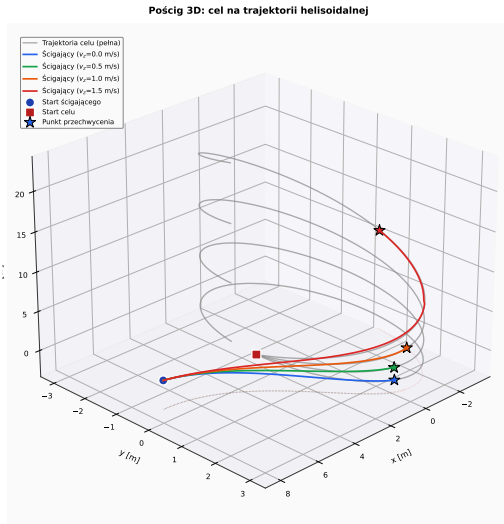
Helisa:

$$\vec{t}(t) = [r \cos(\omega t), r \sin(\omega t), v_z \cdot t]^T$$

Parametry: pozycja startowa ścigającego (8, 0, 2), $r = 3$ m, $\omega = 0.4$ rad/s, $v_p = 2.5$ m/s

Prędkość pionowa v_z [m/s]	Czas przechwycenia [s]	Długość trajektorii [m]
0.0 (okrąg)	3.29	8.23
0.5	3.25	8.14
1.0	3.71	9.21
1.5	7.39	17.07
2.0	15.49	38.45

Pościg 3D – helisa – wizualizacja (Direct Pursuit)



Legenda:

- Start ścigającego
- Start celu
- ★ Punkty przechwycenia
- Pełna trajektoria celu
- Trajektoria ścigającego

Obserwacje:

Szare linie pokazują pełne trajektorie helisy (celu), a gwiazdki wskazują gdzie nastąpiło przechwycenie.

Krzywe Lissajous w 3D (Direct Pursuit)

Trajektoria:

$$\vec{t}(t) = [A_x \sin(\omega_x t), A_y \sin(\omega_y t), A_z \sin(\omega_z t)]^T$$

Warunek wystarczający przechwycenia:

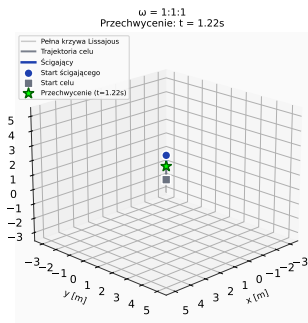
$$v_p > v_{t,\max} = \sqrt{(A_x \omega_x)^2 + (A_y \omega_y)^2 + (A_z \omega_z)^2}$$

$\omega_x : \omega_y : \omega_z$	$v_{t,\max}$	Czas [s]	Długość [m]
1 : 1 : 1	5.20	1.22	3.66
1 : 2 : 1	7.35	> 30	89.97
2 : 3 : 5	18.49	> 30	89.79

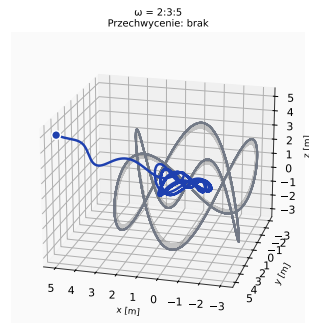
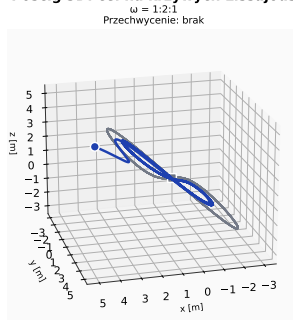
Wniosek: Dla 1:1:1 cel oscyluje liniowo – przechwycenie w punkcie zwrotnym ($v_t = 0$).

Krzywe Lissajous – wizualizacja (Direct Pursuit)

Parametry: start ścigającego (5,5,5), start celu (0,0,0), $A = (3,3,3)$ m, $v_p = 3.0$ m/s



Pościg 3D: cel na krzywych Lissajous



Pościg na sferze S^2 (Direct Pursuit)

Operujemy we **współrzędnych sferycznych** (θ, ϕ) , nie kartezjańskich:

- $\theta \in [0, \pi]$ – kąt polarny (od bieguna północnego)
- $\phi \in [0, 2\pi)$ – kąt azymutalny (długość geograficzna)

Metryka riemannowska:

$$ds^2 = R^2 d\theta^2 + R^2 \sin^2 \theta d\phi^2$$

Odległość kątowa:

$$d_{\text{angular}} = \arccos(\sin \theta_p \sin \theta_t + \cos \theta_p \cos \theta_t \cos(\phi_t - \phi_p))$$

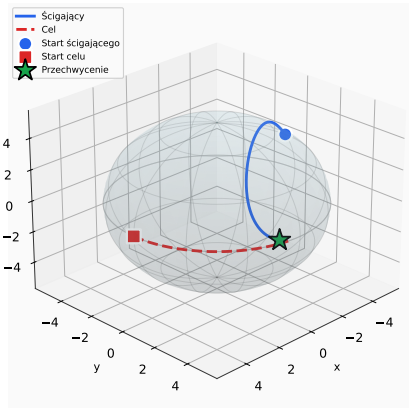
Prędkość v_p/v_t	Czas pościgu [s]	Długość trajektorii [rad]
1.5	1.04	1.19
2.0	0.83	1.26
3.0	0.60	1.34

$R = 5$, $\dot{\theta}_t = 0.1$, $\dot{\phi}_t = \pi/4$ rad/s. Start: $d_0 = 1.87$ rad.

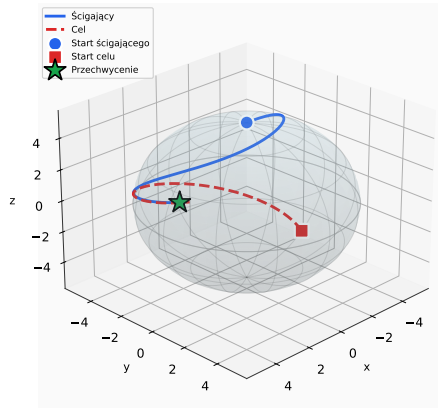
Pościg na sferze – wizualizacja (Direct Pursuit)

Pościg na sferze S^2 ($R=5.0$, współrzędne sferyczne)

Prosty przypadek (cel na równiku)



Złożony przypadek ($v_p/v_t \approx 1.2$)



Pościg na torusie (Direct Pursuit)

Operujemy we **współrzędnych toroidalnych** (u, v) :

- $u \in [0, 2\pi)$ – kąt wokół głównej osi torusa
- $v \in [0, 2\pi)$ – kąt wokół rurki

Metryka riemannowska:

$$ds^2 = (R + r \cos v)^2 du^2 + r^2 dv^2$$

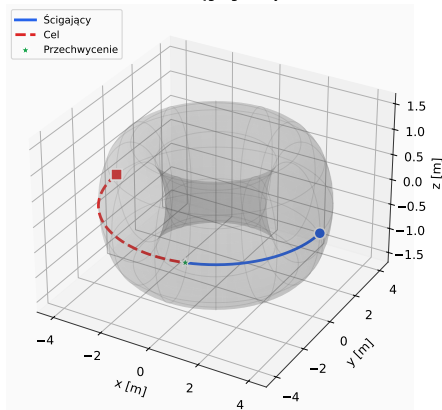
Prędkość v_p	Stosunek v_p/v_t	Czas [s]
1.5	1.21	4.64
2.0	1.62	11.03
2.5	2.02	7.16
3.0	2.43	5.50

$$R = 3, r = 1.2, \omega_u = 0.4, \omega_v = 0.3 \text{ rad/s.}$$

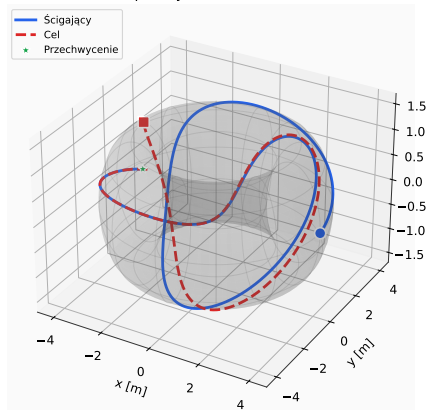
Pościg na torusie – wizualizacja

Pościg na torusie ($R=3.0$ m, $r=1.2$ m)

Cel na okręgu głównym



Cel spiralny ($\omega u=0.3$, $\omega v=0.5$)



Podejście ciągłe vs. dyskretne

Ciągłe (`scipy.integrate.solve_ivp`):

- Adaptacyjny krok czasowy
- Kontrola błędu (tolerancje względne i bezwzględne)
- Detekcja zdarzeń (przechwycenie celu)

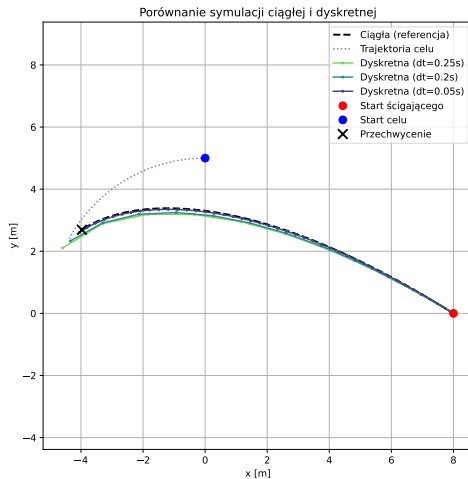
Dyskretne (iteracja z krokiem Δt):

$$\vec{p}_{k+1} = \vec{p}_k + \Delta t \cdot \vec{v}_p(\vec{p}_k, \vec{t}_k)$$

Im mniejszy Δt , tym trajektoria bliższa rozwiązaniu ciągłemu.

Ciągłe vs. dyskretne – wizualizacja

Parametry: $v_p = 6.0$ m/s, $r = 5$ m, $\omega = 0.5$ rad/s



Technologie:

- Python 3.12+
- NumPy – operacje wektorowe
- SciPy – `solve_ivp` (Runge-Kutta z adaptacyjnym krokiem)
- Matplotlib – wizualizacje 2D/3D

Struktura modułów:

- `common/` – klasy bazowe, symulacja
- `d2/` – strategie 2D
- `d3/` – strategie 3D
- `dn/` – n-wymiarowe
- `sphere/` – sfera S^2
- `torus/` – torus

Porównanie strategii:

- **Direct Pursuit** – najkrótsze trajektorie, prosty w implementacji
- **Proportional Navigation** ($N \in [3, 4]$) – najlepszy kompromis efektywność/stabilność
- **Constant Bearing** – spirale logarytmiczne, najmniej efektywny

Wymiar przestrzeni:

- Czas przechwycenia **niezależny od wymiaru** (dla ruchu jednoosiowego)
- Zweryfikowano numerycznie dla $n \leq 20$

Geometria riemannowska (sfera S^2 , torus):

- Pościg we współrzędnych krzywoliniowych (θ, ϕ) i (u, v)
- Metryka definiuje odległość geodezyjną i kierunek pościgu
- Na torusie: nieliniowa zależność czasu od prędkości ścigającego

Dziękujemy za uwagę!